Deep Reinforcement Learning

Dynamic Programming

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

- Policy evaluation
- Policy improvement
- Policy iteration
- Value iteration
- Asynchronous DP
- Generalized policy iteration

What is Dynamic Programming (DP)?

Dynamic Programming (**DP**) refers to two central factors:

- *Dynamic*: a sequential or temporal problem structure
- *Programming*: mathematical optimization, i.e., an algorithmic / a numerical solution

Further characteristics:

- DP is a collection of algorithms to solve MDPs and neighboring problems.
  - We will **focus only on finite MDPs**.
  - In case of continuous action/state space: apply quantization.
- Use of value functions to organize and structure the search for an optimal policy.

Bellman's optimality principle (1)

We here consider finite state and action spaces $\mathcal{S}$ and $\mathcal{A}$. In the last lecture, we have learned about the *return*, the *value function* and the *Bellman equation*

$$
\begin{aligned}
g_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \\
V^\pi(s_t) &= \mathbb{E}_\pi\left[g_t | s_t\right] = \mathbb{E}\left[r_{t+1} | s_t\right] + \gamma \mathbb{E}_\pi\left[V(s_{t+1}) | s_t\right], \\
V^\pi(s_t) &= \mathbb{E}_\pi\left[r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t\right]. \qquad (1)
\end{aligned}
$$

Theorem: Bellman's principle of optimality (Bellman 1954)
An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

In other/modern words: "the tail of an optimal sequence is optimal for the tail subproblem" (Bertsekas 2019)

Bellman's optimality principle (2)

From this, we derived the *Bellman optimality equation* (Bellman 1954):

$$\begin{aligned} V^*(s) &= \max_a \mathbb{E}\left[r_{t+1} + \gamma V^*(s_{t+1}) | s_t = s, a_t = a\right] \\ &= \max_a \sum_{s_{t+1} \in \mathcal{S}} p(s_{t+1} | s_t = s, a_t = a) \left[r_{t+1} + \gamma V^*(s_{t+1})\right]. \end{aligned} \tag{2}$$

Similarly, we can derive an optimal Q-function:

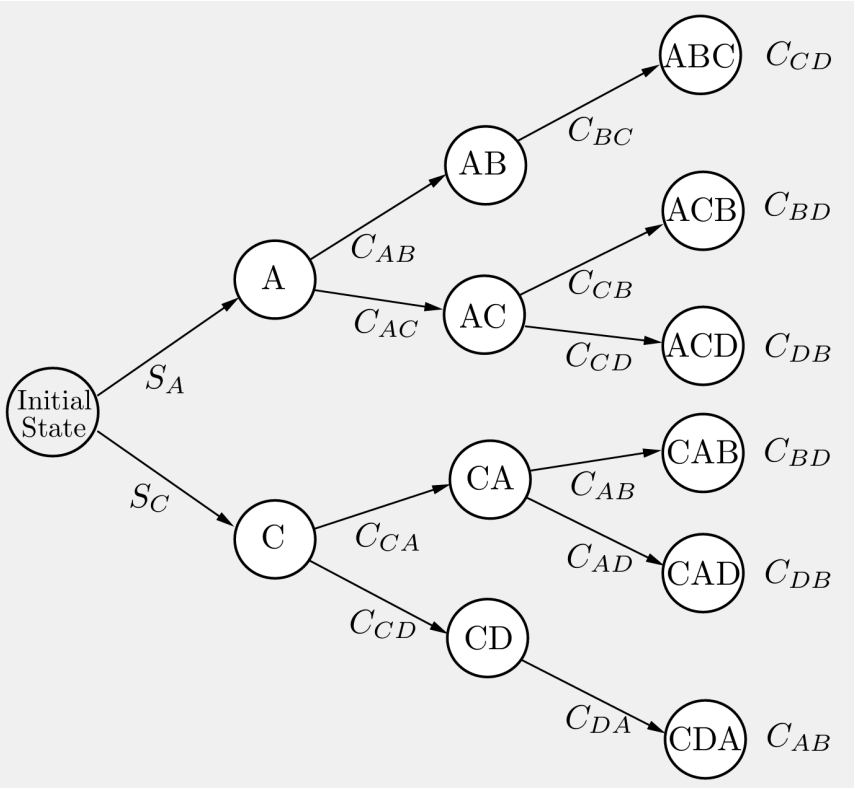
$$\begin{aligned} Q^*(s, a) &= \mathbb{E}\left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \middle| s_t = s, a_t = a\right] \\ &= \sum_{s_{t+1} \in \mathcal{S}} p(s_{t+1} | s_t = s, a_t = a) \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a')\right]. \end{aligned} \tag{3}$$

Central idea of DP: turn these equations into algorithmic assignments.

A DP example from production (1) (Bertsekas 2019)

- to produce a certain product, four operations (denoted by A, B, C, and D) must be performed on a certain machine.
- We assume that operation B can be performed only after operation A has been performed, and operation D can be performed only after operation C has been performed. *(Thus the sequence $CDAB$ is allowable but the sequence $CDBA$ is not.)*
- The setup cost C_{mn} for passing from any operation m to any other operation n is given.
- There is also an initial startup cost S_A or S_C for starting with operation A or C, respectively.
- The cost of a sequence is the sum of the setup costs associated with it; for example, the operation sequence $ACDB$ has cost

$$S_A + C_{AC} + C_{CD} + C_{DB}.$$



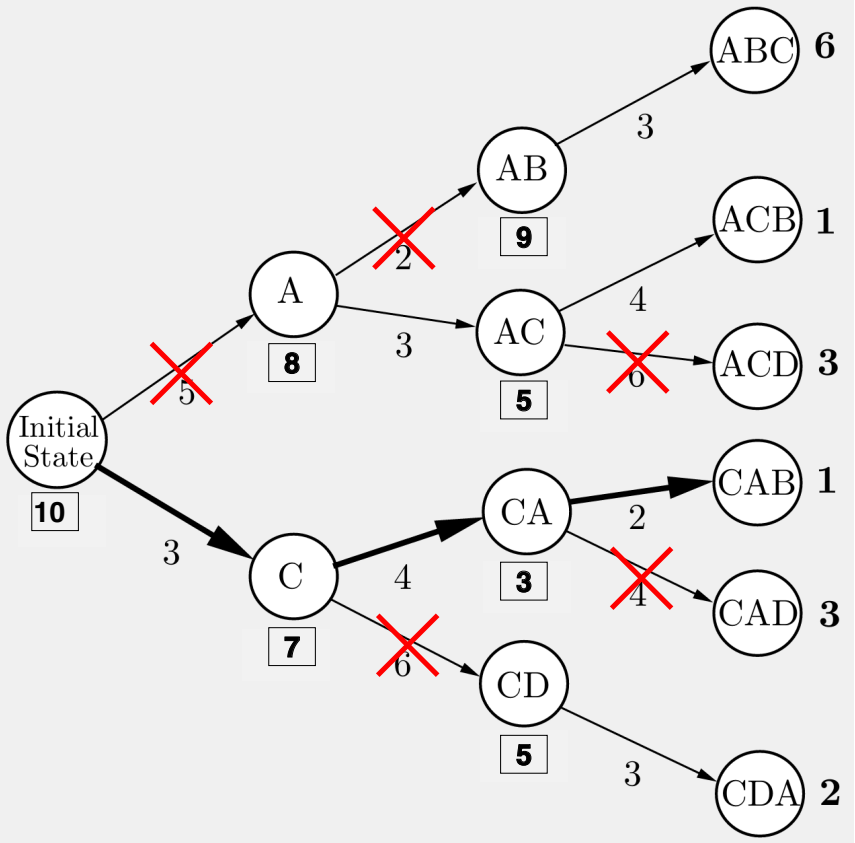
Question: In this example, what is (in the context of RL)

- the state s?

- the action a (and action set $\mathcal{A}(s)$)?

A DP example from production (2) (Bertsekas 2019)

- According to the principle of optimality, the "tail" portion of an optimal schedule must be optimal.
- For example, suppose that the optimal schedule is $CABD$.
- Then, having scheduled first C and then A, it must be optimal to complete the schedule with BD rather than with DB.
- With this in mind, we solve
  - all possible tail subproblems of length two,
  - then all tail subproblems of length three,
  - and finally the original problem that has length four.
  - *(Subproblems of length one are trivial because there is only one unscheduled operation.)*



Policy evaluation

The prediction problem

Let's start with the *prediction problem*: For a given policy π, how do we compute V^π?

Let's revisit (1):

$$V^\pi(s) = \mathbb{E}_\pi\left[r + \gamma V^\pi(s')|s\right] = \sum_{a\in\mathcal{A}} \pi(a|s) \sum_{s'\in\mathcal{S}} p(s'|s,a)\left[r + \gamma V^\pi(s')\right] \tag{4}$$

- If the dynamics p are known, (4) is a system of $|S|$ linear equations in $|S|$ unknowns (the $V^\pi(s)$, $s \in \mathcal{S}$) $\Rightarrow$ Solution is straightforward, if tedious.
- Iterative solution method:
  - Consider a sequence of approximate value functions $V_0, V_1, V_2, \ldots$
  - Choose V_0 arbitrarily (except that the terminal state, if any, must be given value 0).
  - Each successive approximation is obtained by using the Bellman equation (4) as an update rule.

Existence and uniqueness of V^π are guaranteed as long as either $\gamma < 1$ or eventual termination is guaranteed from all states under the policy π.

Iterative policy evaluation

Use the Bellman equation (4) as an update rule:

$$V_{k+1}(s) = \mathbb{E}_\pi \left[r + \gamma V_k(s') | s \right] = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s,a) \left[r + \gamma V_k(s') \right] \tag{5}$$

- $V_k = V^\pi$ is a fixed point for this update rule: (5) is satisfied for V^π.
- The sequence $\{V_k\}$ converges to V^π as $k \to \infty$ (under the same conditions guaranteeing existence of V^π).
- This algorithm is called **iterative policy evaluation**:
  - For each state s, replace the old value of s with a new value ...
  - ... obtained from the old values at the successor states s' and the expected immediate rewards.
  - This is called an *expected update*.

- *Simplest implementation*: keep two copies of V (i.e., V_k and V_{k+1}), then sweep over $\mathcal{S}$.

In-place iterative policy evaluation

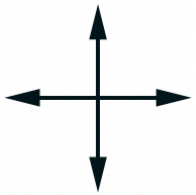
A more memory-efficient version (that tends to converge faster): in-place updates!

Algorithm: In-place iterative Policy Evaluation for estimating $V \approx V^\pi$
Input: policy π
Parameters: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(terminal) = 0$, $\Delta = \infty$
While ($\Delta > \theta$):
    for $s \in \mathcal{S}$:
        $v = V(s)$
        $V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s,a) \left[r + \gamma V(s')\right]$
        $\Delta = \max(\Delta, |v - V(s)|)$

Example: 4×4 gridworld (Sutton and Barto 1998)

- All fields get $r = -1$, except for the terminal gray fields, where $r = 0$.
- $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ ("leaving" $\Rightarrow$ no movement).
- $\pi(\cdot|s) = [0.25, 0.25, 0.25, 0.25]^\top \forall s \in \{1, \dots, 14\}$.



actions

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

Value function for different iterates k:

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 0$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 1$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 2$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 3$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 10$

0.0	-14.	-20.	-22
-14.	-18.	-20.	-20
-20.	-20.	-18.	-14
-22.	-20.	-14.	0.(

$k = \infty$

Policy improvement

Policy and value iteration

Asynchronous DP

Generalized policy iteration

Summary / what you have learned

References

Bellman, Richard. 1954. “The Theory of Dynamic Programming.” *Bulletin of the American Mathematical Society* 60 (6). American Mathematical Society (AMS): 503–15. doi:10.1090/s0002-9904-1954-09848-8.

Bertsekas, Dimitri. 2019. *Reinforcement Learning and Optimal Control*. Vol. 1. Athena Scientific.

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.